

Rootwell — API Reference: Every Call, Resiliency, and Security

Every HTTP call this system makes, internal and external, with what protects it today and what honestly doesn't yet. Written at the same level of detail as `docs/TECHNICAL_GUIDE.md` so gaps are stated plainly — "here's what's missing and the concrete trigger to add it" is a stronger interview answer than pretending everything is hardened.

1. Internal API call map

All internal calls are synchronous HTTP + JSON (no message queue, no async/event-driven hop anywhere yet — see §3 for what that costs).

1a. Frontend → Gateway (the only browser-facing boundary)

Method & path	Purpose
<code>POST /session</code>	Start a session
<code>GET /session/{sid}</code>	Full state (chat, symptoms, causes, recommendations)
<code>POST /session/{sid}/message</code>	Send a chat turn
<code>POST /session/{sid}/add-item / /remove-item</code>	Accept/reject a suggested symptom or cause
<code>POST /session/{sid}/advance-to-causes</code>	Move symptom → cause collection
<code>POST /session/{sid}/analyze</code>	Run the full recommendation pipeline
<code>POST /session/{sid}/email/request / /email/confirm</code>	Verification-gated email export
<code>POST /session/{sid}/end</code>	Explicit purge
<code>GET /healthz</code>	Liveness

`services/gateway/main.py` is a thin 1:1 proxy — every route above just calls `_forward(method, path, body)` against the orchestrator with the same body, and re-raises the orchestrator's real status code (`HTTPException(status_code=resp.status_code, detail=resp.text)`) rather than collapsing every failure into a generic 500. That matters: the frontend can distinguish "session not found" (404) from "bad step transition" (400) from "rate limited" (429) instead of seeing one opaque error for all of them.

1b. Gateway → Orchestrator

Same paths, `/session/*` → `/sessions/*` (plural), otherwise identical request/response bodies — the gateway adds no transformation, only CORS and rate limiting (see §4).

1c. Orchestrator → the 7 agents + email (the fan-out, one call per pipeline stage)

Triggered only by `POST /sessions/{sid}/analyze`, in this exact sequence (`services/orchestrator/main.py`'s `analyze()`), each a blocking await — the next call doesn't start until the previous one returns:

1. `POST agent-mapping:/mapping/analyze` — `{symptom_ids}` → `{imbalances, reasoning}`
2. `POST agent-retrieval:/retrieval/candidates` — `{symptom_ids, imbalances}` → `{candidates}`
3. `POST agent-safety:/safety/evaluate` — `{candidates, profile}` → `{verdicts}`
4. `POST agent-scoring:/scoring/rank` — `{symptom_ids, candidates, verdicts}` → `{ranked}`
5. `POST agent-explanation:/explanation/generate` — `{candidates, ranked, verdicts}` → `{recommendations}`

Plus, outside `/analyze`:

- `GET agent-intake:/intake/greeting` — once, at `POST /sessions`
- `POST agent-intake:/intake/symptom-turn` / `/intake/cause-turn` — every chat message during collection
- `POST email:/email/verify` — at `/email/request`
- `POST agent-reporting:/reporting/compile` then `POST email:/email/send` — at `/email/confirm`, in that order (report must be compiled before it can be sent)

Why sequential, not parallel: stages 2–5 each depend on the previous stage's output (retrieval needs mapping's imbalances; safety needs retrieval's candidate list; scoring needs safety's verdicts; explanation needs scoring's ranks) — this is a genuine data-dependency chain, not an arbitrary choice. There's no parallelizable fan-out to exploit here today.

1d. Agents → Knowledge services (Tier 1 reads)

Caller	Call	Notes
agent-intake	GET knowledge-toxicology:/symptoms	Fetches once per process, cached in a module-level <code>_catalog_cache</code> — not per-request
agent-mapping	GET knowledge-toxicology:/symptoms/{id}	One call per symptom id , in a loop — see §3 for the N+1-shaped cost this implies
agent-retrieval	GET knowledge-botanical:/herbs?symptom_id=...	One call per symptom id, in a loop
agent-retrieval	GET knowledge-compound:/compounds?ids=a,b,c	Batched — all needed compound ids in one call, the one place this pattern is already avoided
agent-safety	GET knowledge-rules:/rules	Fetches the whole rule set, filters in-process (dataset is tiny; see <code>docs/TECHNICAL_GUIDE.md</code> §1 on why this doesn't need pagination yet)

Known inefficiency, honestly flagged: `agent-mapping` and `agent-retrieval` call their knowledge service once per symptom id sequentially rather than batching (the way `agent-retrieval` → `compound` already does with `?ids=`). At ~1-5 symptoms per session this is invisible; it's the same *shape* of problem the N+1 Hibernate question (`mit-lincoln-lab-technical-qa.md` Q29) tests — worth being able to say out loud: "I'd add a batched `?symptom_ids=a,b,c` endpoint the same way `compound` already works, the fix is mechanical, it just hasn't been needed yet at this data volume."

2. External API calls

Only two third-party services are ever called, and both are **optional at runtime** — see `docs/TECHNICAL_GUIDE.md` §1's mock-mode discussion for why that's a deliberate architecture property, not just a nice-to-have.

2a. Anthropic (Claude) — `shared/shared/llm.py`

- **Called from:** `agent-intake` (symptom/cause turn NLU), `agent-mapping` (reasoning summary), `agent-explanation` (per-herb recommendation text)
- **Auth:** `ANTHROPIC_API_KEY` bearer token, read once at process start via `os.environ["ANTHROPIC_API_KEY"]` inside `_get_client()` — never logged, never echoed in any response body
- **Call shape:** `AsyncAnthropic().messages.create(model=..., system=..., messages=[{"role": "user", "content": ...}])` — one-shot completion, no streaming, no conversation history sent to Anthropic (each call is stateless; the actual chat history lives only in Tier 2 Redis)
- **Timeout:** `ANTHROPIC_TIMEOUT_SECONDS` (default 20s), passed to `AsyncAnthropic(timeout=...)` — **fixed**; previously fell back to the SDK's own default with no explicit bound, which meant a slow Anthropic response could hold a `symptom-turn / analyze` request open indefinitely.
- **Fallback on absence/failure:** `complete_or_none()` returns `None` if `ANTHROPIC_API_KEY` is unset (mock mode) — and, **as of this hardening pass**, also returns `None` if the key is set but the call itself throws (network error, rate limit, malformed response, or a timeout): the call is wrapped in a try/except that logs a warning and falls back to the same deterministic template mock mode already uses. Mock mode used to only cover "no key configured," not "key configured but the call failed at runtime" — that asymmetry is now closed on this path. The equivalent gap still exists on the Resend side (§2b).

2b. Resend (transactional email) — `services/email/main.py`

- **Called from:** `email` service only, two endpoints: verification-code send (`/email/verify`) and final report send (`/email/send`)
- **Auth:** `RESEND_API_KEY` bearer token; `RESEND_FROM_ADDRESS` is the only user-influenceable-adjacent field, and it's a deployment-time env var, never request input
- **Timeout:** `10.0s` on both calls (`httpx.AsyncClient(timeout=10.0)`)
- **Fallback on absence:** `MOCK_MODE = not bool(RESEND_API_KEY)` — logs the fully rendered email to stdout instead of sending
- **Fallback on failure (key present, call fails):** same gap as Anthropic above — `resp.raise_for_status()` on the send call is not wrapped, so a transient Resend outage becomes a 500 to the user at the exact moment they're trying to export their results and have their session purged. Given the export step immediately triggers `purge_session` (see `docs/ARCHITECTURE.md` §4 step 4), **this is the single highest-value place in the whole system**

to add a retry — losing a session's data because of one transient email-provider blip, right as the user asked to save it, is the worst-case failure mode in this app.

3. Resiliency: what exists, what's honestly missing

Concern	Current state	Gap / trigger to add it
Timeouts	Every internal call has an explicit <code>httpx</code> timeout (10s for agent↔knowledge/agent calls, 30s for gateway↔orchestrator and orchestrator↔agent, since those wrap the <i>whole</i> multi-hop analyze chain). Anthropic SDK call now has one too (<code>ANTHROPIC_TIMEOUT_SECONDS</code> , default 20s — see §2a).	Closed
Retries / backoff	Still none, anywhere — every failed internal call (<code>resp.raise_for_status()</code>) propagates as an exception immediately. The Anthropic call gained a <i>fallback</i> , not a <i>retry</i> (§2a): one failed attempt degrades straight to the mock template rather than retrying.	Add real retry/backoff (2-3 attempts, exponential) specifically around the Resend send call (§2b), which is the one external call left with neither a retry nor a fallback — not around internal agent-to-agent calls, where a failure usually means a real bug (bad request shape), and retrying a broken request 3 times just triples the useless load
Circuit breaking	None	Not yet justified at this scale (7 agents, one call chain, no cascading-failure history) — would become worth it once/if any one knowledge service starts seeing real latency variance under load; premature today

Concern	Current state	Gap / trigger to add it
Idempotency	<code>GET</code> calls are naturally idempotent (safe to retry blindly). <code>POST /sessions</code> creates a new session each call — not idempotent by design (every retry should start fresh, there's no "resume" semantics). <code>POST /email/send</code> is effectively idempotent against double-charge-style bugs: the verification token is deleted (<code>await r.delete(_verify_key(...))</code>) on first successful use, so a retried send with the same token+code fails cleanly with "token expired or not found" rather than sending twice.	The one place idempotency actually matters most (email send) already has it, structurally, via single-use token deletion — worth being able to explain <i>why</i> that pattern works even without a dedicated idempotency-key header (<code>mit-lincoln-lab-technical-qa.md</code> Q36)
Backpressure / overload protection	Only the gateway's per-IP rate limiter (§4) — a blunt, request-count-based limit, not a true backpressure mechanism (no queue depth signal, no load-shedding based on downstream latency)	Fine for a single-instance local system; would need a real signal (e.g., orchestrator call latency, or a queue if one existed) once this runs multi-instance
Dead-letter / lost-work handling	None — there's no queue to have a DLQ in front of. Every failure is synchronous and visible to the caller immediately (no silent background failure mode exists today)	This is actually a point <i>in favor</i> of the current fully-synchronous design: a DLQ exists to catch failures a fire-and-forget/async pipeline would otherwise lose silently. Since nothing here is fire-and-forget, there's nothing that can fail invisibly — the trade is "no async pipeline" for "no DLQ needed," not "a DLQ is missing"
Self-healing (Tier 1 cache)	Every <code>knowledge-*</code> service reloads Tier 1 data from <code>seed/data/*.json</code> on its own startup (idempotent <code>load_seed()</code>) — a container restart automatically repopulates it, no manual intervention	Already resilient to Redis Tier 1 data loss, by construction
Session data loss on Redis	Real gap: Tier 2 has no persistence backing it beyond Redis's own (unconfigured) durability — a	Acceptable today given the TTL/hard-purge lifecycle is

Concern	Current state	Gap / trigger to add it
restart (Tier 2)	Redis restart mid-session loses that session's chat/symptom/cause state entirely	already designed around "this is meant to be ephemeral," but worth stating plainly rather than implying Tier 2 is durable

4. Security: boundary by boundary

4a. Browser ↔ Gateway (the only externally-reachable boundary)

- **CORS:** `CORSMiddleware` allow-lists origins from `CORS_ALLOW_ORIGINS` (default `http://localhost:3000`), not a wildcard — the frontend's origin is the only one that can call the gateway from a browser context.
- **Rate limiting:** per-client-IP, sliding one-minute window, via `Redis INCR + EXPIRE` (`gateway:ratelimit:{ip}:{minute}` key), default 60 req/min, 429 past that. This is the gateway's one piece of abuse-prevention logic and it's coarse (IP-based, trivially bypassed by rotating IPs) — appropriate for a local/demo system, not sufficient alone for a public launch.
- **No authentication.** There is no login, no account system — a session id (`uuid4().hex`, 128 bits of randomness) *is* the bearer credential (see `docs/ARCHITECTURE.md` §4). This is a scope decision for an anonymous-by-design app, and it means: anyone who obtains a session id (e.g., via a leaked URL/log line) can read and act on that session until it's purged or expires. No JWT/OAuth2 anywhere in the system yet — see `docs/TECHNICAL_GUIDE.md` §6 for where that would go if real accounts are ever added.

4b. Gateway ↔ Orchestrator ↔ Agents ↔ Knowledge services

- **No auth, no mTLS between any internal hop today.** Trust is entirely "if you're on the Docker Compose network, you're trusted" — the perimeter (gateway) is the only enforcement point. This is the same trade-off called out in `docs/TECHNICAL_GUIDE.md` §4/§6: reasonable at "13 containers on one host," and exactly the boundary a service mesh (Istio, `mit-lincoln-lab-technical-qa.md` Q8-11) or application-layer JWT passed hop-to-hop would need to close before a genuinely multi-tenant or zero-trust deployment.

- **Input validation:** every service boundary validates the incoming body against a Pydantic model before any handler code runs — malformed requests are rejected with a `422` before they can reach business logic, which is the FastAPI-native version of "validate at every trust boundary."
- **Redis has no `requirepass` configured.** Any container on the compose network can read/write any key, Tier 1 or Tier 2, with no credential. Acceptable for an isolated local Docker network with nothing else attached to it; would be a real finding in any deployment where Redis's network boundary is less tightly controlled.

4c. Email export anti-abuse (`services/email/main.py`)

Specifically designed so "email me this" can't become an open relay:

- **Verification-code gate:** sending requires a prior `/email/verify` → a 6-digit code delivered to the target address, confirmed via `/email/confirm` before `/email/send` will act. Codes expire after 10 minutes (`VERIFY_TTL_SECONDS`) and are single-use (deleted on consumption — see idempotency note in §3).
- **Per-recipient rate limit:** max 3 verification requests per address per hour (`RATE_LIMIT_MAX_PER_WINDOW` / `_WINDOW_SECONDS`) — stops someone from using this endpoint to spam-verify (and thus spam) an address that isn't theirs, even without ever completing a send.

4d. Fixed finding: unescaped user text in the emailed HTML report

The original `services/agents/reporting/main.py` (now deleted — this service is Java, see `docs/ARCHITECTURE.md` §7) built the HTML export by directly f-string-interpolating symptom labels, cause labels, and chat message text into HTML tags — e.g. `f"<li>{s.get('label')}</li>"` and `f"<p>...{msg['text']}</p>"` — **with no HTML-escaping**. Cause labels and chat text are free-form user input (typed directly, or matched/echoed by the LLM in real mode), so this was the same category as OWASP's injection class applied to HTML context (`mit-lincoln-lab-technical-qa.md` Q42): a `<script>` or `<img onerror=...>` payload in a user's message would have been written verbatim into the HTML email body. The realistic blast radius was always limited (only ever affects the recipient's own inbox rendering their own submitted data — no cross-user exposure, since Tier 2 data is per-session-private), but it was a genuine gap, not a hypothetical one.

Fixed during the Java rewrite.

`services/agents/reporting/src/main/java/app/rootwell/reporting/ReportingService.java` runs every interpolated field (symptom/cause labels, herb names, safety notes, chat text)

through `HtmlUtils.htmlEscape()` before it reaches the `html` output — the class's own doc comment references this exact finding and confirms it's resolved there. The plain-`text` export needs no escaping (not markup). Worth naming in an interview as a real "found a security gap, then closed it as part of an unrelated migration" story, including the honest coda: this doc and `CLAUDE.md` briefly described the finding as still-open after the fix had already landed, until that inconsistency was caught by reading `ReportingService.java` directly rather than trusting the doc.

4e. Secrets handling

- `ANTHROPIC_API_KEY` / `RESEND_API_KEY` / `RESEND_FROM_ADDRESS` are read from environment variables only, sourced from a gitignored `.env` (`.env` and `.env.local` are both in `.gitignore`) — never committed, never logged, never returned in any response body (verified: no `main.py` echoes these values back to a caller).
- No secret ever appears in a Redis key or value — verification tokens and codes are app-generated random values (`secrets.token_urlsafe` , `secrets.randbelow`), not the API keys themselves.

5. Summary: the honest one-paragraph version

Every external dependency is optional and fails soft when *absent* (mock mode); the Anthropic path now also fails soft when *present-and-failing* (`shared/shared/11m.py` added a timeout plus a try/except-to-mock-template fallback on runtime failure). The same asymmetry still exists on the Resend side — no retry/backoff or fallback if a configured key starts failing at send time — and that's now the single most valuable resiliency gap to describe if asked "what would you harden first," precisely because it fires at the exact moment a user is trying to save their session, right before purge. Security today is perimeter-only (gateway CORS + rate limit + email verification gate), nothing internal — appropriate for a single-host local system, and the concrete triggers to close each remaining gap (mTLS/Istio, JWT, Redis auth, retry/backoff on the Resend call) are each named

above rather than left implicit. The report's HTML-escaping gap (\$4d) has since been fixed, during the Java rewrite of `agent-reporting`.